Practical exercises in Security
Cryptography and ACs

1 Cryptography: basic operations

The openssl framework (`apt-get install openssl`) can be useful for generating and signing certificates. You can use the following instructions:

- `openssl genrsa -out <file.pem> <size>`: generates an RSA key pair of *size* bits.
  What should the value of *size* be? For which scenarios?
- `openssl rsa -in <file.pem> -text –noout` generates a detailed screen output of the file content storing an RSA key.
- `openssl rsa -in <file.pem> -des3 -out <file.pem>`: encrypts the private key using the DES3 algorithm. Do you prefer DES, 3DES or IDEA?
- `openssl rsa -in <file.pem> -pubout -out <pub_file.pem>`: stores the public part in a separate file.
- `openssl enc <-algo> -in <original> -out <encrypted>`: encrypts original using the specified algorithm (run `openssl enc --help` to view the list of possibilities).
- `openssl enc <-algo> -in <encrypted> -d -out <result>`: for decryption.
- `openssl rsautl -sign -in <input> -inkey <key.pem> -out <sign>`: for signing a message.
- `openssl rsautl -verify -in <signature> -inkey <key> -out <hash>`: for checking a signature.
- `openssl dgst <algo> -out <output> <input>`: for hashing a file; *algo* is the encryption algorithm.

Exercise 1 – Generate your public key.

- You can encrypt a file via:

```
openssl rsautl -pubin -inkey <pub_key.pem> -in <input_file> -encrypt
-out <output_file>
```

To decrypt a file, using the following command:

```
- openssl rsautl -inkey <priv_key.pem> -in <input_file> -decrypt
-out <output_file>
```

Exercise 2 – *The encrypted.txt file (http://ares.insa-lyon.fr/~ftheoley/cours/secu/openssl/encrypte.txt) has been encrypted, using a password, with the AES algorithm in 256 bits in OFB mode. The password encrypted*

with my public key is available at http://ares.insa-lyon.fr/~ftheoley/cours/secu/openssl/cle_sym_cryptee.

In addition, my private key is available at *http://ares.insa-lyon.fr/~ftheoley/cours/secu/openssl/rsa_key.pem*, and has been encrypted with the password *erf6h*. Can you decrypt the message?

Exercise 3 – *You can now try signing a file (any file).*
What operations do you have to perform before signing a file? What are the steps necessary for verifications? Are the public/private keys used for signature and verification?

2. Certificates

2.1. Generating an authority certificate

To generate an AC (show curiosity and look at the content of this script):
- `/usr/lib/ssl/misc/CA.sh -newca`: generates the tree structure of files used
- `/usr/lib/ssl/misc/CA.sh -newreq`: generates a key and a signature query
- `usr/lib/ssl/misc/CA.sh -sign`: signs the certificate.

If everything is automatic, how do you specify the duration of validity of a certificate or the size of the key, for example?

2.2. Useful commands

The *openssl* framework for certificates:
- `openssl req [-config req.cnf] -new -key LoginKey.pem -out LoginReq.pem`: generates a signature query for the key *LoginKey.pem*, basing itself on the configuration file *req.cnf*. The query is stored in *LoginReq.pem*.
- `openssl req -in req.pem -text -noout`: lists the content of the query (for information purposes).
- `openssl x509 -CA caCert.pem -Cakey caKey.pem -CAserial serial`
- `-req -in loginReq.pem -out loginCert.pem`: validates the signature request
- `openssl verify -CAfile caCert.pem loginCert.pem`: checks the validity of the signature
- `openssl pkcs12 -export -in loginCert.pem -inkey loginKey.key`

- `-certfile demoCA/cacert.pem -out loginCert.p12`: exports the public key, the private key and the certificate of the AC to one single pkcs12 file (useful for Windows clients).

Exercise 4 – *You are now going to be able to generate your certification authority. You will then manually create a key pair and the associated certificate. You can enter the information items required for the certificate manually or by specifying a req.cnf configuration file. All that remains now is to have the key signed by the certification authority. You will check that the final certificate does indeed include the right information. At the same time, validate the signature using your AC's certificate.*

Exercise 5 – *Generate a second certificate using the script CA.sh.*

Exercise 6 – *You can now use your certificate to sign your mail messages, for example. The Mozilla mail client enables you to import certificates in pkcs12 mode. However, your correspondent must accept the certification authority having signed your certificate.*
Similarly, you can encrypt an email if you have your recipient's public key, i.e. his/her certificate.

Important note 1: *You are asked to generate at least 2 certificates signed by your certification authority. In addition, all the data on these certificates and the AC must be saved, because they will be used for a future practical exercise.*